ICPC Team Notebook

2026

Carlos E. R. Oliveira • Kaique A. de Oliveira • Mateus A. Mizael

Sumário

1 Matemática **1**
1.1 Combinatória . . . 1
1.1.1 Stars and Bars . . . 2
1.1.2 Princípio da Inclusão-Exclusão (PIE) . . . 2
1.2 Coeficientes Binomiais (Combinatória Otimizada) . . . 2
1.3 Permutação . . . 2
1.4 Fast Pow . . . 3
1.5 Inverso Modular . . . 3
1.6 Teorema Chinês dos Restos (CRT) . . . 3
1.7 Totient de Euler . . . 4
1.8 Linearidade da Expectativa e Variáveis Indicadoras . . . 4
1.9 Fast Fourier Transform (FFT) . . . 4
1.10 Exponenciação de Matrizes . . . 5
2 Geometria **6**
2.1 Interseção de Segmentos . . . 6
3 Arrays **6**
3.1 Contagem de Inversões . . . 6
4 Estruturas de Dados **6**
4.1 Fenwick Tree . . . 6
4.2 Merge Sort Tree . . . 7
5 Grafos **7**
5.1 MST — Kruskal . . . 7
5.2 MST — Prim . . . 8
5.3 Bellman-Ford . . . 8
5.4 Floyd-Warshall . . . 9
6 Programação Dinâmica **9**
6.1 LIS — Longest Increasing Subsequence . . . 9
6.2 LCS — Longest Common Subsequence . . . 9
6.3 Maximum Sum 2D . . . 10
7 Strings **10**
7.1 KMP . . . 10
7.2 Z-Function . . . 10
8 Misc **11**
8.1 Big Integer . . . 11
8.1.1 Utilitários . . . 11
8.1.2 Soma . . . 11
8.1.3 Subtração . . . 11
8.1.4 Multiplicação por Inteiro . . . 12
8.1.5 Divisão por Inteiro . . . 12
8.1.6 Módulo por Inteiro . . . 12

1. Matemática

1.1 Combinatória

Combinações distintas escolhendo k elementos dentre n elementos, sem importar a ordem:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Propriedade útil:

$$\binom{n}{k} = \binom{n}{n-k}$$

1.1.1 Stars and Bars

Utilizado para distribuir n objetos idênticos em k caixas distintas.

- **Caso 1 (Variáveis não-negativas, $x_i \geq 0$):** O número de soluções inteiras para a equação $x_1+x_2+\cdots+x_k = n$ é:

$$\binom{n+k-1}{k-1}$$

- **Caso 2 (Variáveis estritamente positivas, $x_i \geq 1$):** O número de soluções inteiras para a equação $x_1 + x_2 + \cdots + x_k = n$ é:

$$\binom{n-1}{k-1}$$

1.1.2 Princípio da Inclusão-Exclusão (PIE)

Calcula o tamanho da união de conjuntos ou conta elementos que evitam um conjunto de restrições.

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1}|A_1 \cap \cdots \cap A_n|$$

Implementação clássica via bitmask (ideal para $n \leq 20$):

```
// Exemplo geral para contar elementos usando PIE via bitmask
ll total_union = 0;
int n_sets = 15; // numero de condicoes/conjuntos
for (int mask = 0; mask < (1 << n_sets); mask++) {
    ll size_intersection = get_intersection_size(mask);
    if (__builtin_popcount(mask) % 2 == 1) total_union += size_intersection;
    else total_union -= size_intersection;
}
// Se o problema pede elementos que nao violam nenhuma condicao: Total - total_union
```

1.2 Coeficientes Binomiais (Combinatória Otimizada)

Válido para MOD primo.

Complexidade: Pré-computação em $O(N \log(\text{MOD}))$ e consultas em $O(1)$.

```
ll fact[MAXN], invFact[MAXN];

void precompute() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN - 1] = modInverse(fact[MAXN - 1], MOD);
    for (int i = MAXN - 2; i >= 1; i--)
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
}

// iterativamente em O(r): res = res * (n - r + i) / i; i = [1, k]
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

1.3 Permutação

Número de maneiras de escolher e ordenar k elementos dentre n:

$$P(n,k) = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1)$$

Complexidade: $O(k)$.

```
ll P(int n, int k) {
    ll res = 1;
    for (int i = 0; i < k; i++) res *= n - i;
    return res;
}
```

1.4 Fast Pow

Complexidade: $O(\log b)$.

```
ll binpow(ll a, ll b, ll mod) {
    a %= mod;
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

1.5 Inverso Modular

x é o inverso modular de a módulo m se:

$$ax \equiv 1 \pmod{m}$$

Existe se e somente se:

$$\gcd(a, m) = 1$$

Se m é primo:

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

Complexidade: $O(\log m)$.

Requer: m primo e $a \not\equiv 0 \pmod{m}$.

```
ll modInverse(ll a, ll mod) {
    return binpow(a, mod - 2, mod);
}
```

1.6 Teorema Chinês dos Restos (CRT)

Resolve sistemas de congruências lineares: $x \equiv a_i \pmod{m_i}$ para $i = 0, \ldots, n-1$.

O algoritmo abaixo funciona mesmo se os módulos m_i **não forem coprimos**. Retorna $\{x, M\}$ onde a solução geral é $x \pmod{M}$ ou $\{-1, -1\}$ se não houver solução.

Complexidade: $O(n \log(\min(m)))$.

```
// Retorna o gcd e calcula x, y tais que a*x + b*y = gcd(a, b)
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return a; }
    ll d = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

// Retorna {x, M} tal que x == A[i] (mod M[i]) ou {-1, -1} se inconsistente
pair<ll, ll> crt(vector<ll> A, vector<ll> M) {
    ll a1 = A[0], m1 = M[0];
    for (size_t i = 1; i < A.size(); i++) {
        ll a2 = A[i], m2 = M[i];
        ll p, q;
        ll g = extgcd(m1, m2, p, q);
        if ((a2 - a1) % g != 0) return {-1, -1}; // Sem solucao
```

```
        __int128 mod = (__int128)m1 * (m2 / g);
        __int128 k = (a2 - a1) / g;
        __int128 m2_g = m2 / g;
        p = (p % m2_g + m2_g) % m2_g;
        __int128 diff = (__int128)p * (k % m2_g) % m2_g;

        __int128 x = (__int128)a1 + (__int128)m1 * diff;
        a1 = (x % mod + mod) % mod;
        m1 = mod;
    }
    return {a1, m1};
}
```

1.7 Totient de Euler

$\varphi(n)$ = quantidade de inteiros em $[1, n]$ coprimos com n.

Se:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

então:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Complexidade: $O(\sqrt{n})$.

```
ll phi(ll n) {
    ll res = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}
```

1.8 Linearidade da Expectativa e Variáveis Indicadoras

A esperança (valor esperado) da soma de variáveis aleatórias é sempre igual à soma das suas esperanças, **independentemente de haver dependência entre elas**:

$$E\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} E[X_i]$$

Para uma variável indicadora I_A de um evento A (onde $I_A = 1$ se A ocorre, 0 caso contrário):

$$E[I_A] = P(A)$$

1. Defina X como a variável total que você quer calcular o valor esperado.
2. Tente expressar $X = \sum_{i=1}^{n} X_i$, onde cada X_i é o menor subproblema/evento possível.
3. Transforme X_i em uma variável indicadora $I_i \in \{0, 1\}$.
4. Lembre-se: $E[X] = \sum E[I_i] = \sum P(\text{evento } i \text{ ocorrer})$.
5. Se os $P(\text{evento } i)$ forem idênticos por simetria, $E[X] = n \cdot P(\text{evento 1})$.

1.9 Fast Fourier Transform (FFT)

O tamanho do vetor resultante deve ser ajustado para a /textbfpróxima potência de 2.

Complexidade: $O(N \log N)$

```cpp
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) for (auto &x : a) x /= n;
}

vector<ll> multiply(vector<ll>& a, vector<ll>& b) {
    vector<cd> fa(all(a)), fb(all(b));
    int n = 1;
    while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n); fb.resize(n);

    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    fft(fa, true);

    vector<ll> result(n);
    for (int i = 0; i < n; i++) result[i] = round(fa[i].real());
    return result;
}
```

1.10 Exponenciação de Matrizes

Complexidade: $O(N^3 \log P)$

```cpp
using Matrix = vector<vector<ll>>;

Matrix multiply(const Matrix &A, const Matrix &B) {
    int n = A.size();
    Matrix C(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (A[i][k] == 0) continue;
            for (int j = 0; j < n; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
        }
    }
    return C;
}

Matrix power(Matrix A, ll p) {
    int n = A.size();
    Matrix res(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // Matriz Identidade
    while (p > 0) {
        if (p & 1) res = multiply(res, A);
```

```
        A = multiply(A, A);
        p >>= 1;
    }
    return res;
}
```

2. Geometria

2.1 Interseção de Segmentos

Complexidade: $O(1)$.

```
struct Point {
    long long x, y;
};
long long cross (Point p1, Point p2, Point p3) {
    return ((p2.x - p1.x) * (p3.y - p1.y)) - ((p2.y - p1.y) * (p3.x - p1.x));
};
int sgn (long long val) {
    if (val > 0) return 1;
    if (val < 0) return -1;
    return 0;
};
long long overlap (long long a1, long long a2, long long b1, long long b2) {
    if (a1 > a2) swap(a1, a2);
    if (b1 > b2) swap(b1, b2);
    return max(a1, b1) <= min(a2, b2);
};
bool intersect (Point a, Point b, Point c, Point d) {
    int cp1 = sgn(cross(a, b, c));
    int cp2 = sgn(cross(a, b, d));
    int cp3 = sgn(cross(c, d, a));
    int cp4 = sgn(cross(c, d, b));
    if (cp1 == 0 && cp2 == 0 && cp3 == 0 && cp4 == 0)
        return overlap(a.x, b.x, c.x, d.x) && overlap(a.y, b.y, c.y, d.y);
    return (cp1 != cp2) && (cp3 != cp4);
};
```

3. Arrays

3.1 Contagem de Inversões

Um par (i, j) é uma inversão se:

$$i < j \quad \text{e} \quad a_i > a_j$$

Complexidade: $O(N \log N)$.

```
long long count_inversions (vector<int>& a) {
    // Compressao de Coordenadas
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    SegTree st(vals.size()); // Segtree para soma e update = 1
    long long inv = 0;

    for (int x : a) {
        int rank = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
        inv += st.query(rank + 1, (int)vals.size() - 1);
        st.update(rank, 1);
    }
    return inv;
}
```

4. Estruturas de Dados

4.1 Fenwick Tree

Suporta soma de prefixo e atualização pontual.

Atualização: $O(\log N)$

Consulta de prefixo: $O(\log N)$

Soma em $[l, r]$**:** $O(\log N)$

Usa índices de 1 a N.

```
int n;
vector<long long> bit;
void init (int size) {
    n = size;
    bit.assign(n + 1, 0);
}
void add (int i, long long x) {
    for (; i <= n; i += i & -i) bit[i] += x;
}
long long sum (int i) {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
long long query (int l, int r) {
    return sum(r) - sum(l - 1);
}
```

4.2 Merge Sort Tree

Permite consultas de ordem em intervalos usando `lower_bound` / `upper_bound`.

Construção: $O(N \log N)$.

Consulta: $O(\log^2 N)$.

```
int n;
vector<vector<int>> tree;
void build (const vector<int>& a, int p, int l, int r) {
    if (l == r) {
        tree[p] = {a[l]};
        return;
    }
    int m = (l + r) / 2;
    build(a, p * 2, l, m);
    build(a, p * 2 + 1, m + 1, r);

    merge(tree[p * 2].begin(), tree[p * 2].end(),
          tree[p * 2 + 1].begin(), tree[p * 2 + 1].end(), back_inserter(tree[p]));
}
int query (int ql, int qr, int x, int p, int l, int r) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr)
        return lower_bound(tree[p].begin(), tree[p].end(), x) - tree[p].begin();
    int m = (l + r) / 2;
    return query(ql, qr, x, p * 2, l, m) + query(ql, qr, x, p * 2 + 1, m + 1, r);
}
```

O exemplo acima conta elementos $< x$ no intervalo $[l, r]$.

Para contar elementos $\leq x$, use `upper_bound`.

5. Grafos

5.1 MST — Kruskal

Complexidade: $O(E \log E)$.

```
using Edge = tuple<long long, int, int>;
long long kruskal (int n, vector<Edge> edges) {
    priority_queue<Edge, vector<Edge>, greater<Edge>> pq;
    for (const auto& [w, u, v] : edges) pq.push({w, u, v});
    DSU dsu(n);
    long long totW = 0;
    int used = 0;
```

```
    while (!pq.empty()) {
        auto [w, u, v] = pq.top(); pq.pop();
        if (!dsu.unite(u, v)) continue;
        totW += w;
        used++;
        if (used == n - 1) break;
    }
    if (used != n - 1)
        return -1;
    return totW;
}
```

5.2 MST — Prim

Adequado para grafos densos representados por matriz de adjacência.

Complexidade: $O(V^2)$.

```
long long prim (const vector<vector<long long>>& graph) {
    int n = graph.size();
    vector<long long> minEdge(n, INF);
    vector<bool> used(n, false);
    long long totW = 0;
    minEdge[0] = 0;

    for (int i = 0; i < n; i++) {
        int curr = -1;
        for (int j = 0; j < n; j++)
            if (!used[j] && (curr == -1 || minEdge[j] < minEdge[curr])) curr = j;
        if (curr == -1 || minEdge[curr] == INF) return -1;
        used[curr] = true;
        totW += minEdge[curr];
        for (int k = 0; k < n; k++)
            if (!used[k] && graph[curr][k] < minEdge[k]) minEdge[k] = graph[curr][k];
    }
    return totW;
}
```

5.3 Bellman-Ford

Encontra menores caminhos a partir de uma origem em grafos com pesos negativos.

Também detecta ciclos negativos alcançáveis a partir da origem.

Complexidade: $O(VE)$.

```
using Edge = tuple<long long, int, int>;
pair<vector<long long>, bool> bellmanFord (int n, const vector<Edge>& edges, int origin) {
    vector<long long> dist(n, INF);
    dist[origin] = 0;
    for (int i = 0; i < n - 1; i++) {
        bool changed = false;
        for (auto [u, v, w] : edges) {
            if (dist[u] == INF) continue;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (const Edge& edge : edges) {
        if (dist[u] == INF) continue;
        if (dist[u] + w < dist[v]) return {dist, true};
    }
    return {dist, false};
}
```

5.4 Floyd-Warshall

Encontra os menores caminhos entre **todos os pares** de vértices.

Também funciona com arestas negativas, desde que não existam ciclos negativos.

Complexidade: $O(V^3)$.

Memória: $O(V^2)$.

```
void floydWarshall (vector<vector<long long>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; k++) {
        for (int u = 0; u < n; u++) {
            if (dist[u][k] == INF) continue;
            for (int v = 0; v < n; v++) {
                if (dist[k][v] == INF) continue;
                dist[u][v] = min(dist[u][v], dist[u][k] + dist[k][v]);
            }
        }
    }
}
```

Após o algoritmo, se:

$$dist[v][v] < 0$$

para algum vértice v, existe um ciclo negativo.

6. Programação Dinâmica

6.1 LIS — Longest Increasing Subsequence

Complexidade: $O(N \log N)$.

```
int LIS (const vector<int>& values) {
    vector<int> tails;
    for (int value : values) {
        auto position = lower_bound(tails.begin(), tails.end(), value);
        if (position == tails.end()) tails.push_back(value);
        else *position = value;
    }
    return tails.size();
}
```

`tails[i]` representa o menor valor possível para o último elemento de uma LIS de tamanho $i + 1$.

Para subsequência **não decrescente**, use `upper_bound`.

6.2 LCS — Longest Common Subsequence

Complexidade: $O(NM)$.

```
int LCS (const string& first, const string& second) {
    int fLen = first.size();
    int sLen = second.size();
    vector<vector<int>> dp(fLen + 1, vector<int>(sLen + 1, 0));
    for (int i = 1; i <= fLen; i++) {
        for (int j = 1; j <= sLen; j++) {
            if (first[i - 1] == second[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[fLen][sLen];
}
```

Transição:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & a_i = b_j \\ \max(dp[i-1][j], dp[i][j-1]), & a_i \neq b_j \end{cases}$$

6.3 Maximum Sum 2D

Complexidade: $O(N^2M)$.

Para uma matriz $N \times M$:

```
long long maximumSum2D (const vector<vector<long long>>& matrix) {
    int n = matrix.size();
    int m = matrix[0].size();
    long long ans = LLONG_MIN;

    for (int i = 0; i < n; i++) {
        vector<long long> columnSum(m, 0);
        for (int j = i; j < n; j++) {
            for (int column = 0; column < m; column++)
                columnSum[column] += matrix[j][column];
            long long currentSum = 0;
            for (long long value : columnSum) {
                currentSum = max(value, currentSum + value);
                ans = max(ans, currentSum);
            }
        }
    }
    return ans;
}
```

7. Strings

7.1 KMP

Prefix Function: $O(M)$.

Busca: $O(N)$.

```
vector<int> prefixFunction (const string& pattern) {
    int pLen = pattern.size();
    vector<int> pref(pLen);
    for (int i = 1; i < pLen; i++) {
        int matched = pref[i - 1];
        while (matched > 0 && pattern[i] != pattern[matched])
            matched = pref[matched - 1];
        if (pattern[i] == pattern[matched]) matched++;
        pref[i] = matched;
    }
    return pref;
}
int kmp (const string& text, const string& pattern) {
    vector<int> pref = prefFunction(pattern);
    int pLen = pattern.size();
    for (int txtIdx = 0, matched = 0; txtIdx < text.size(); txtIdx++) {
        while (matched > 0 && text[txtIdx] != pattern[matched])
            matched = pref[matched - 1];
        if (text[txtIdx] == pattern[matched]) matched++;
        if (matched == pLen) return txtIdx - pLen + 1;
    }
    return -1;
}
```

7.2 Z-Function

Complexidade: $O(N)$.

```
vector<int> zFunction (const string& text) {
    int n = text.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && text[z[i]] == text[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
```

```
    }
    return z;
}
```

8. Misc

8.1 Big Integer

8.1.1 Utilitários

```
string stripLeadingZeros (const string& number) {
    int firstNonZero = number.find_first_not_of('0');
    if (firstNonZero == string::npos) return "0";
    return number.substr(firstNonZero);
}
```

8.1.2 Soma

Assume que ambos os números são positivos.

Complexidade: $O(N)$.

```
string addBig (const string& first, const string& second) {
    string result;
    int carry = 0;
    int i = first.size() - 1;
    int j = second.size() - 1;
    while (i >= 0 || j >= 0 || carry) {
        int digitSum = carry;
        if (i >= 0) digitSum += first[i--] - '0';
        if (j >= 0) digitSum += second[j--] - '0';
        result.push_back('0' + digitSum % 10);
        carry = digitSum / 10;
    }
    reverse(result.begin(), result.end());
    return stripLeadingZeros(result);
}
```

8.1.3 Subtração

Assume que $first \geq second$.

Complexidade: $O(N)$.

```
string subtractBig (const string& first, const string& second) {
    string result;
    int borrow = 0;
    int i = first.size() - 1;
    int j = second.size() - 1;
    while (i >= 0) {
        int digit = (first[i] - '0') - borrow;
        if (j >= 0)
            digit -= second[j--] - '0';
        if (digit < 0) {
            digit += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        result.push_back('0' + digit);
        i--;
    }
    reverse(result.begin(), result.end());
    return stripLeadingZeros(result);
}
```

8.1.4 Multiplicação por Inteiro

Complexidade: $O(N)$.

```
string multiplyByInt (const string& number, int multiplier) {
    string result;
    long long carry = 0;
    for (int i = number.size() - 1; i >= 0; i--) {
        long long product = (number[i] - '0') * 1LL * multiplier;
        product += carry;
        result.push_back('0' + product % 10);
        carry = product / 10;
    }
    while (carry > 0) {
        result.push_back('0' + carry % 10);
        carry /= 10;
    }
    reverse(result.begin(), result.end());
    return stripLeadingZeros(result);
}
```

8.1.5 Divisão por Inteiro

Complexidade: $O(N)$.

```
pair<string, int> divideByInt (const string& number, int divisor) {
    string quotient;
    long long remainder = 0;
    for (char digit : number) {
        remainder = remainder * 10 + (digit - '0');
        quotient.push_back('0' + remainder / divisor);
        remainder %= divisor;
    }
    return {stripLeadingZeros(quotient), remainder};
}
```

8.1.6 Módulo por Inteiro

Complexidade: $O(N)$.

```
int modString (const string& number, int mod) {
    long long remainder = 0;
    for (char digit : number)
        remainder = (remainder * 10 + (digit - '0')) % mod;
    return remainder;
}
```